

LLM7 (GPT)

def inspecção - rede (res - nominal, res - medida):
 Tolerância - abs = res - nominal * 0.08
 Devia = res - medida - res - nominal
 Devia - PCT = (Devia / res - nominal) * 100

IF - Tolerância - abs <= Devia <= Tolerância - abs:
 return "Sólo aprovado"

else:
 return (F"Se não aprovado: {F}"
 F"Devia: {Devia + 2F} MPa {F}"
 F"Devia (%): {Devia - PCT: + 2F} %")

def controle velocidade (V atual, V alvo):
 IF V atual < V - alvo * 1.10:
 return "Velocidade alta - risco de processamento - interrompa"

def V atual < V - alvo * 0.90:
 return "Velocidade baixa - gargalo na produção"

else:
 return "Velocidade OK"

Testes
 Print("==== Teste Parte A ====")
 Print (inspecção - rede (120, 90))
 Print ("==== Teste Parte B ====")
 Print (controle de velocidade (145, 120))

1. Algoritmo empacotamento - biblioteca

Indica:
 // descrição de variáveis
 Indica t, dias, horas, minutos, segundos, resto1, resto2

// Inicializa
 Escreva ("Digite a tempo total do material em segundos")
 Leia (t)

// Processamento
 IF 86400 segundos < 24 horas * 3600 segundos
 dias <- t / 86400 // divisão
 resto1 <- t - dias * 86400 // resto que sobra dos dias (em segundos)
 horas <- resto1 / 3600 // divisão inteira para obter os horas restantes
 resto2 <- resto1 % 3600 // resto que sobra em horas (em segundos)
 minutos <- resto2 / 60
 segundos <- resto2 % 60

Escreva (dias, " dias, ", horas, " horas, ", minutos, " minutos, ", segundos)

Fim

5. Fundamentos

Paralela

Escreva "Informe a resistência nominal (MPa):"
 Leia Resistência - nominal

Escreva "Informe a medida (MPa):"
 Leia Resistência - medida

Tolerância - PCT < 3
 Tolerância - abs <= Resistência nominal * 0.08

Devia <= Resistência medida - Resistência nominal
 Devia - PCT = (Devia / Resistência nominal) * 100

Se Devia = - Tolerância - abs E (Devia > Tolerância)
 Escreva "Sólo aprovado"
 Escreva "Devia: ", Devia, " MPa"
 Escreva "Devia (%): ", Devia - PCT, "%"

Fim Se

// Parte B
 Escreva "Informe a velocidade alvo (m/min):"
 Leia Velocidade - alvo

Escreva "Informe a velocidade atual (m/min):"
 Leia Velocidade - atual

Se Velocidade - atual > Velocidade - alvo * 1.10 então
 Escreva "Velocidade alta - risco de processamento - interrompa"

Senão Se Velocidade - atual < Velocidade - alvo * 0.90 então
 Escreva "Velocidade baixa - gargalo na produção"

Senão Escreva "Velocidade OK"

Fim

5. Fundamentos

Paralela

Escreva "Informe a resistência nominal (MPa):"
 Leia Resistência - nominal

Escreva "Informe a medida (MPa):"
 Leia Resistência - medida

Tolerância - PCT < 3
 Tolerância - abs <= Resistência nominal * 0.08

Devia <= Resistência medida - Resistência nominal
 Devia - PCT = (Devia / Resistência nominal) * 100

Se Devia = - Tolerância - abs E (Devia > Tolerância)
 Escreva "Sólo aprovado"
 Escreva "Devia: ", Devia, " MPa"
 Escreva "Devia (%): ", Devia - PCT, "%"

Fim Se

// Parte B
 Escreva "Informe a velocidade alvo (m/min):"
 Leia Velocidade - alvo

Escreva "Informe a velocidade atual (m/min):"
 Leia Velocidade - atual

Se Velocidade - atual > Velocidade - alvo * 1.10 então
 Escreva "Velocidade alta - risco de processamento - interrompa"

Senão Se Velocidade - atual < Velocidade - alvo * 0.90 então
 Escreva "Velocidade baixa - gargalo na produção"

Senão Escreva "Velocidade OK"

Fim

1. Algoritmo conversão-tempo

iniciando
 // declaração de variáveis
 letura t, dias, horas, minutos, segundos, resto1, resto2

// Início
 Escreva "Digite o tempo total em minutos em segundos:"
 leia(t)
 // Processamento
 // 86400 segundos = 24 horas * 3600 segundos
 dias ← t / 86400 // divisão
 resto1 ← t % 86400 // Resto que sobra dos dias (em segundos)
 horas ← resto1 / 3600 // divisão inteira para obter as horas
 resto2 ← resto1 % 3600 // resto que sobra das horas (em segundos)
 minutos ← resto2 / 60
 segundos ← resto2 % 60

Escreva (dias, " dias, ", horas, " horas, ", minutos, " minutos, ", segundos)

Fim

LLM 2 (Gemini)

base_falada = float(input("Preço da falada (R\$): "))
 alg_PI = float(input("PI (R\$): "))
 alg_Kms = float(input("ICMS (%): "))
 alg_Pis = calcula_falada(input("PS (R\$): "))

PI = base_falada * (alg_PI / 100)
 base_ICMS = base_falada + PI
 ICMS = base_ICMS * (alg_Kms / 100)
 base_PC = base_ICMS + ICMS
 PS_calculado = base_PC * (alg_Pis / 100)
 base_final = base_PC + PS_calculado

tempo_total = ((base_final - preco_inicial) / preco_unitario) * 60

Print("PI: ", PI)
 Print("ICMS: ", ICMS)
 Print("PS: ", PS_calculado)
 Print("base_PC: ", base_PC)
 Print("base_final: ", base_final)
 Print("tempo_total: ", tempo_total)

Comparação dos códigos dos LLMs:

LLM 1 (GPT): Código em Python bem estruturado e funcional. O GPT tem uma ótima utilização de variáveis e código claro que facilita a leitura e a execução.

LLM 2 (Gemini): Código em Python mais simples e direto. Lembre-se de que o código "limpo" é mais fácil de entender e manter.

Conclusão: O GPT é superior em termos de estrutura e clareza, enquanto o Gemini é mais direto e simples.

1. Algoritmo conversão-tempo

iniciando
 // declaração de variáveis
 letura t, dias, horas, minutos, segundos, resto1, resto2

// Início
 Escreva "Digite o tempo total em minutos em segundos:"
 leia(t)
 // Processamento
 // 86400 segundos = 24 horas * 3600 segundos
 dias ← t / 86400 // divisão
 resto1 ← t % 86400 // Resto que sobra dos dias (em segundos)
 horas ← resto1 / 3600 // divisão inteira para obter as horas
 resto2 ← resto1 % 3600 // resto que sobra das horas (em segundos)
 minutos ← resto2 / 60
 segundos ← resto2 % 60

Escreva (dias, " dias, ", horas, " horas, ", minutos, " minutos, ", segundos)

Fim

Pythhon

LLM

Calculo (Preço - Fábricao, alíq - IPT, alíq - ICMS, alíq - PIS - Confins):

$$IPT = \text{Preço} - \text{Fábricao} \times \text{alíq} - IPT / 100$$

$$\text{base} - \text{ICMS} = \text{Preço} - \text{Fábricao} + IPT$$

$$ICMS = \text{base} - \text{ICMS} \times \text{alíq} - ICMS / 100$$

$$\text{base} - PC = \text{base} - \text{ICMS} + ICMS$$

$$PIS - Confins = \text{base} - PC \times \text{alíq} - PIS - Confins / 100$$

$$\text{Preço} - \text{final} = \text{base} - PC + PIS - Confins$$

$$\text{Grp} - \text{tbl} = ((\text{Preço} - \text{final} - \text{Preço} - \text{Fábricao}) / \text{base} - \text{ICMS}) \times 100$$

Print "n" === Demonstração ===

Print F "Preço de fábrica" RS (Preço - Fábricao: 10.2 F)"

Print F "IPT:" RS (IPT: 10.2 F)"

Print F "Base ICMS:" RS (base - ICMS: 10.2 F)"

Print F "ICMS:" RS (ICMS: 10.2 F)"

Print F "Base PIS / Confins:" RS (base - PC: 10.2 F)"

Print F "PIS / Confins:" RS (PIS - Confins: 10.2 F)"

Print F "Preço final:" RS (Preço - final: 10.2 F)"

Print F "Grp - tbl:" (Grp - tbl: 10.2 F)"

data

/ /

S T Q Q S S D

Python

LLM 1 (GP+)

 $R1 = \text{Float}(\text{input}(\text{digite } R1 \text{ (ohms): " "}))$ $R2 = \text{Float}(\text{input}(\text{digite } R2 \text{ (ohms): " "}))$ $R3 = \text{Float}(\text{input}(\text{digite } R3 \text{ (ohms): " "}))$ $V_{\text{fonte}} = \text{Float}(\text{input}(\text{digite tensão de fonte (V): " "}))$ $R_x = (R3 \times R2) / R1$ $R_{\text{total}} = (R1 + R_x) \times (R2 + R3) / (R1 + R2 + R3 + R_x)$ $I_{\text{total}} = V_{\text{fonte}} / R_{\text{total}}$ $P_{\text{total}} = V_{\text{fonte}} \times I_{\text{total}}$ IF $R1 \times R_x == R2 \times R3$: $\text{equilibrio} = \text{"Ponte equilibrada"}$

else:

 $\text{equilibrio} = \text{"Ponte não equilibrada"}$ $\text{Print}(\text{f" } R_x = (R_x:.2f) \text{ Ohms" })$ $\text{Print}(\text{f" } R_{\text{total}} = (R_{\text{total}}:.2f) \text{ Ohms" })$ $\text{Print}(\text{f" } I_{\text{total}} = (I_{\text{total}}:.4f) \text{ A" })$ $\text{Print}(\text{f" } P_{\text{total}} = (P_{\text{total}}:.4f) \text{ W" })$ $\text{Print}(\text{equilibrio})$

LL M2 (Gmini)

R1 = Float (input ("R1 (Ω): "))

R2 = Float (input ("R2 (Ω): "))

R3 = Float (input ("R3 (Ω): "))

VF = Float (input ("V Fonte (V): "))

$$R_x = (R_3 \times R_2) / R_1$$

equilíbrio / mirror

$$R_{\text{total}} = ((R_1 + R_x) \times (R_2 + R_3)) / (R_1 + R_x + R_2 + R_3)$$

$$I_{\text{total}} = VF / R_{\text{total}}$$

$$P_{\text{total}} = VF \times I_{\text{total}}$$

$$\text{equilíbrio} = (R_1 \times R_x) = (R_2 \times R_3)$$

Print ("- " * 30)

Print (f"Rx: (Rx: .2f) Ω ")

Print (f"R Total: (R Total: .2f) Ω :")

Print (f"I total: (I total: .4f) A:")

Print (f"P total: (P total: .4f) W:")

Print (f"Equilíbrio? (Sim if equilibria else 'no')")

LLM1(GPT)

def classificar - Solo (SPT, cop, carga):

IF SPT < 5 or cop - carga < 10:

return "Inócuo: Solo muito fraco - reforço necessário"

ELIF SPT < 10 OR cop - carga < 50

msg = "Restrito: fundação expulsa recomendada \n"

IF SPT < 10:

msg += "- Motivo: SPT abaixo do limite \n"

IF cop - carga < 150:

msg += "- Motivo: capacidade de carga abaixo do limite"

return msg

ELIF SPT >= 30 and cop - carga >= 300

return "Excelente: Solo de alta capacidade"

ELSE:

return "adequado: Fundação convencional normal"

Testes = [(13, 200), (15, 120), (35, 320)]

for SPT, cop in testes:

Print(f"Entrada: SPT={SPT}, Capacidade={cop} KPa")

Print(classificar - Solo (SPT, Cop))

Código em Python:

~~LLM 1~~ LLM 1 (Gemin):

t = int(input("Digite o tempo total da missão em segundos:"))

((//) e modulo (%))

dias = t // 86400

resto1 = t % 86400

horas = resto1 // 3600

resto2 = resto1 % 3600

minutos = resto2 // 60

segundos = resto2 % 60

Print(f"({dias}) dias, ({horas}) (Minutos:02 d): (Segundos:02 d)")

LLM 2 (GPT):

t = int(input("Digite o tempo em segundos (>=0):"))

Processamento

Dias = t // 86400

resto1 = t % 86400

horas = resto1 // 3600

resto2 = resto1 % 3600

Minutos = resto2 // 60

Segundos = resto2 % 60

Print(f"({dias}|{horas}|{Minutos:02d}):({Segundos:02d})")